

Indo para 2D

Malha · N · J · n · propgeo · GeomGmsh

Lucas S. Campos

Introdução ao MEC / BEM

Indo para 2D

A partir desta aula o laboratório usa o pacote `GeomGmsh` em `codes/geom_gmsh` (Julia + Gmsh; recorte de geometria do `BEM_gmsh`).

Foco: **geometria de contorno** — malha, elementos, jacobiano, normal e integrais só em Γ .

Condições de contorno (o que cada aresta “vale” em T ou q) ficam para o capítulo **Laplace 2D**.

Os trechos Julia são **pedaços do próprio pacote** (`src/`) ou dos exemplos em `scripts/` e `_solu/`.

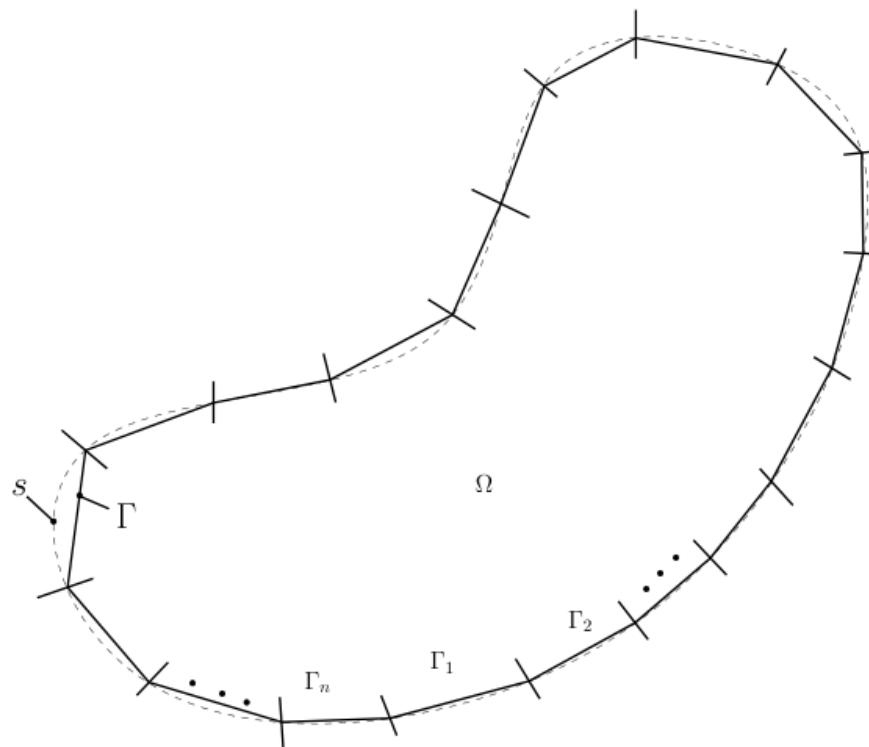
1. Ativar o ambiente GeomGmsh e gerar uma malha de contorno com Gmsh.
2. Entender o elemento descontínuo de ordem p via `discontinuous_nodes_weights` + `shapefun` (Lagrange baricêntrico).
3. Calcular J , tangente e normal como em `format2d`.
4. Obter perímetro, área e centróide com `geometric_props` nas **duas** estratégias: `:radial` (default) e `:divergence`.
5. Rodar os exemplos `scripts/circulo.jl` (2D) e `scripts/toro.jl` (3D).

1. Por que 2D e o que muda em relação ao BEM 1D
2. Ambiente GeomGmsh e Gmsh (geometria, sem CDC)
3. Elemento descontínuo + Lagrange baricêntrico (`Interpolation.jl` / `Input.jl`)
4. Jacobiano, normal (como em `format2d`)
5. `geometric_props` — radial e divergência (`GeometricProperties.jl`)
6. Exemplo: disco unitário
7. Exemplo: toro 3D (`format3d`)
8. Exercícios

Por que “indo para 2D”

No BEM 1D o “contorno” eram dois pontos. Em 2D:

- $\Omega \subset \mathbb{R}^2$ tem fronteira $\Gamma = \partial\Omega$ **unidimensional** (curva fechada, eventualmente com furos);
- a geometria de Γ é aproximada por **elementos de contorno** Γ_e ;
- campo e fluxo no contorno usam interpolação em $\xi \in [-1, 1]$ sobre cada Γ_e ;
- integrais de área em Ω que admitem redução a Γ (radial ou divergência) preparam as identidades do BEM.



Ideia-chave. Malhar só Γ não elimina a necessidade de boa geometria: N_k , J e $\mathbf{n}$ em cada elemento são os tijolos de `geometric_props` e, depois, de H e G .

Na pasta codes/geom_gmsh:

```
julia --project=. -e "using Pkg; Pkg.instantiate()"
julia --project=. scripts/circulo.jl
julia --project=. scripts/toro.jl
```

O pacote Gmsh.jl já é dependência. GUI: gmsh.info.

Geradores (circulo, anel, placa_com_furo, cubo, toro, ...) em src/Meshes.jl: gmsh.initialize() → gerar → datadir(nome * ".msh") → finalize(). Depois format2d / format3d e geometric_props.

Por que Gmsh no BEM (só geometria)

1. Curvas, arcos, splines, furos e sólidos OCC (toro) com orientação controlável.
2. Malha de contorno (curvas) + superfície 3D; volume opcional (pontos internos / orientação).
3. Ordem geométrica alinhada ao grau do elemento (ordem / tipo).
4. O mesmo gerador alimenta Laplace, Poisson e elasticidade no BEM_gmsh completo.

Nesta aula os grupos físicos **nomeiam** pedaços de curva e a superfície do domínio. A convenção "0;T" / "1;q" entra em **Laplace 2D**.

Orientação (crítico)

- Contorno **externo: anti-horário** → normal **para fora** de Ω .
- Contorno de **furo: horário** → normal ainda saindo de Ω .
- $A < 0$ (ou $V < 0$ em 3D) denuncia orientação invertida.

`format2d(; orient=true)` (default) corrige o sinal de **n** com as células 2D da malha de superfície. `placa_com_furo(; reverse_hole=true)` já inverte o laço do furo no Gmsh.

O que format2d faz (Input.jl)

```
# nós de colocação descontínuos = Gauss-Legendre (p+1 pontos)
qsi, wi = discontinuous_nodes_weights(p) # = gausslegendre(p+1)

# geometria isoparamétrica nos nós Equispaced de grau p
Ngeo, dNgeo = shapefun(Equispaced(p), qsi)

# para cada aresta da malha Gmsh, com vértices X:
nodes[idx] .= Ngeo * X           # posições dos nós de campo
dx          = dNgeo * X
J           = norm.(dx)
normal[idx] = tan2normal.(dx ./ J)
L           = abs(dot(J, wi))    # comprimento do elemento
```

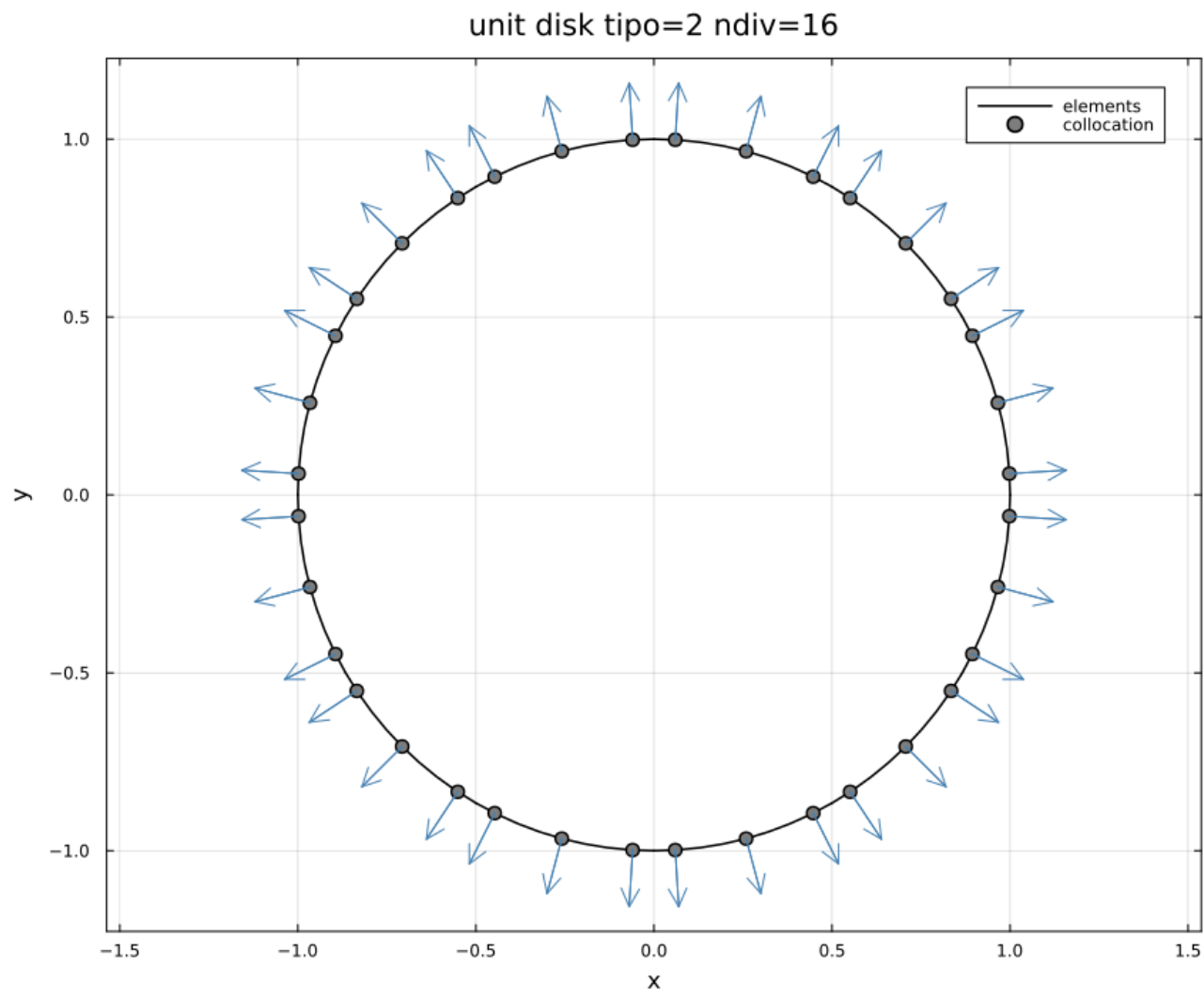
Os **mesmos** shapefun e pesos entram depois em H e G .

```
using GeomGmsh

msh = circulo(; ndiv=16, show=false, nome="ex_circulo", ordem=2)
dad = format2d(msh, Laplace(1.0); tipo=2, pontointerno=false)
g = geometric_props(dad)          # strategy=:radial
println("  perimeter = ", g.perimeter, "  (exact  $2\pi$ )")
println("  area      = ", g.area, "  (exact  $\pi$ )")
println("  centroid  = ", g.centroid, "  (exact 0,0)")
plot_geo(dad; show_normals=true)
```

format2d monta BEMdata. **Nesta aula:** plot_geo e geometric_props, não solve / H_G*.

Disco unitário (plot_geo)



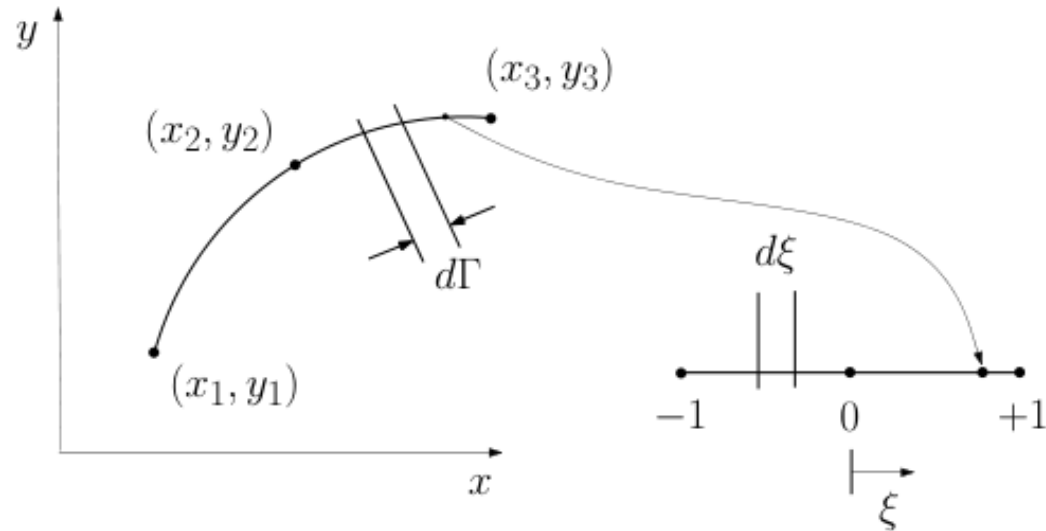
Disco: números (tipo=2, ndiv=16)

Grandeza	Numérico	Exato
perímetro	6.2827	$2\pi \approx 6.2832$
área	3.1411	$\pi \approx 3.1416$
centróide	$\approx (0, 0)$	$(0, 0)$

A malha linear (tipo=1) é um n -ágono regular: $P = 2n \sin(\pi/n) = 2\pi - O(n^{-2})$, $A = (n/2) \sin(2\pi/n) = \pi - O(n^{-2})$.
Com tipo=2 os arcos parabólicos aproximam o círculo e o erro cai bem mais rápido.

Elemento de contorno descontínuo

No GeomGmsh o padrão é **elemento descontínuo**: graus de liberdade de campo **não** são compartilhados nos vértices entre elementos vizinhos (evita conflito de CDC em cantos — detalhe em Laplace 2D).



$$\Gamma_e$$

é a imagem de $\xi \in [-1, 1]$:

$$\mathbf{x}(\xi) = \sum_{k=1}^{p+1} N_k(\xi), \mathbf{x}_k^{(e)}.$$

- **Geometria** da aresta Gmsh: nós Equispaced(p) (vértices + meios, se ordem 2).
- **Campo** (colocação): nós Legendre(p) = Gauss–Legendre em $(-1, 1)$ via `discontinuous_nodes_weights(p)`.

```
function discontinuous_nodes_weights(p::Integer)
    p >= 1 || error("degree must be ≥ 1, got $p")
    return gausslegendre(p + 1)
end
```

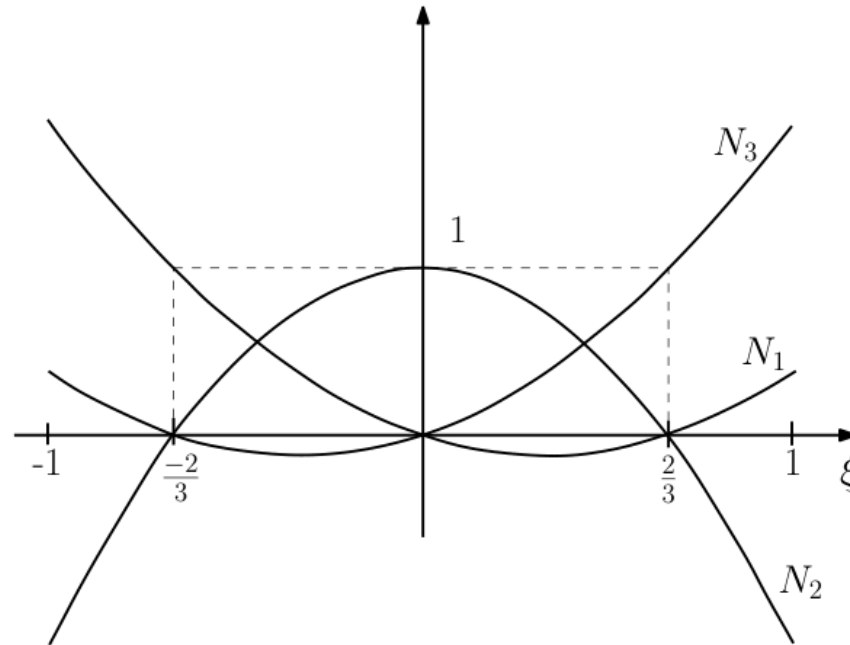
tipo=1,2,3 em `format2d` = grau p do campo.

Pesos (nós x_0):

$$w_i = 1 / \prod_{j \neq i} (x_i - x_j).$$

$$N_i(\xi) = (w_i / (\xi - x_i)) / \sum_j (w_j / (\xi - x_j)).$$

```
function shapefun(poly::AbstractPolynomial, x)
    L = interpolation_matrix(poly, x)    # N(x)
    L, L * poly.Dmat                   # N e dN/dξ
end
```



Dmat (resumo)

$$u'(\xi_k) = \sum_i N_{i'}(\xi_k) u_i \quad \Rightarrow \quad \mathbf{u}' = D\mathbf{u}$$

$$D_{ki} = (w_i/w_k)/(x_k - x_i) \quad (k \neq i), \quad D_{kk} = -\sum_{i \neq k} D_{ki}$$

poly.Dmat é montada **uma vez** por diff_matrix. A diagonal impõe $\sum_i N_{i'} = 0$.

```
L = interpolation_matrix(poly, x)    # N
dN = L * poly.Dmat                  # N_i'(\xi) = \sum_k N_k(\xi) D_ki
```

Dmat **não** é o jacobiano da malha.

```
p = 2
poly = Legendre(p)           # campo descontínuo (nós de Gauss)
xi = nodes(poly)            # nós de colocação
wN = weights(poly)           # pesos *baricêntricos* (não de Gauss)
xi_g, wg = gausslegendre(6)
N, dN = shapefun(poly, xi_g) # size (6 x 3) se p=2

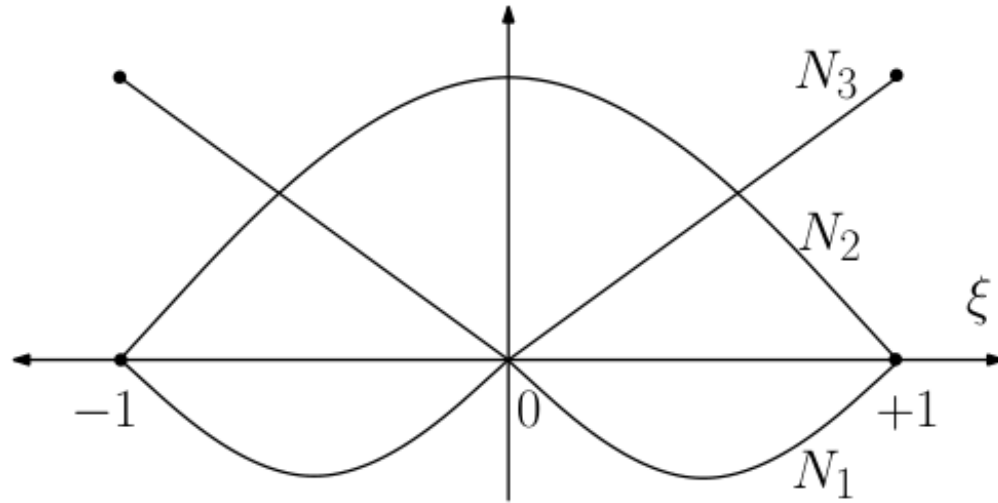
poly_geo = Equispaced(p)     # geometria contínua da aresta
Ngeo, dNgeo = shapefun(poly_geo, xi_g)
```

Partição da unidade (E1):

```
poly = Legendre(3)
N, dN = shapefun(poly, xi)
@assert abs(sum(N) - 1) < 1e-11
@assert abs(sum(dN)) < 1e-9
```

Equispaced(2) = Lagrange clássico

$$N_1 = \xi(\xi - 1)/2, \quad N_2 = 1 - \xi^2, \quad N_3 = \xi(\xi + 1)/2.$$



Usado na **geometria** da aresta Gmsh de ordem 2.

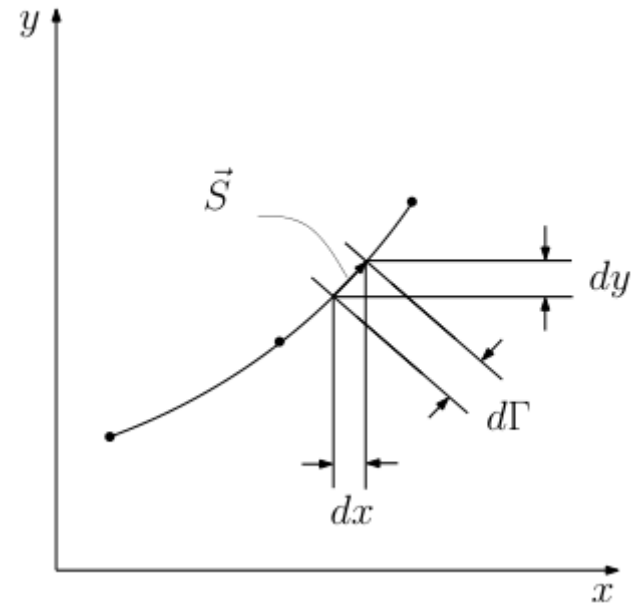
Depois de `format2d`, `dad.element_type` é o polinômio do campo (`Legendre(p)`), `dad.elem_weight` são os pesos de Gauss da colocação, e cada `dad.elements[e]` guarda índices, jacobianos nos nós de colocação e comprimento. `dad.Nodes` / `dad.Normal` são vistas de `collocation`.

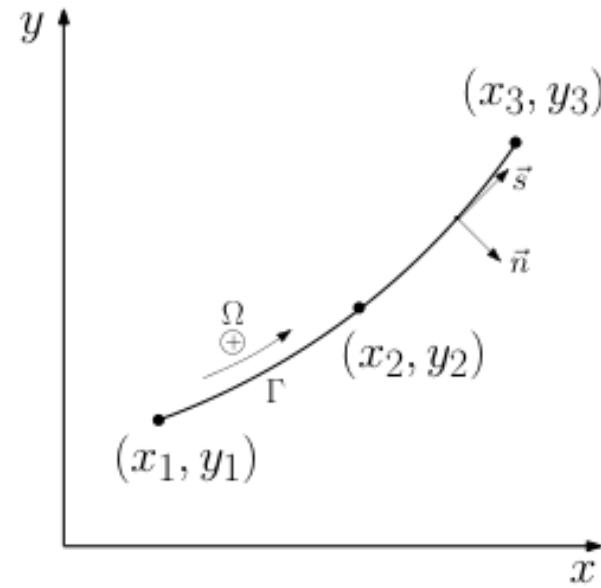
Jacobiano, comprimento e normal

$$\mathbf{x}'(\xi) = \sum_k N_{k'}(\xi), \mathbf{X}_k, \quad J = |\mathbf{x}'|, \quad \mathbf{n} = (y', -x')/J$$

```
dx = dNgeo * X
J = norm.(dx)
normal = tan2normal.(dx ./ J) # (dx,dy) → (dy, -dx)/|·|
L = abs(dot(J, wi))          # ∫ J dξ ≈ Σ J_k w_k
```

Perímetro global: $P = \sum_e L_e$ (g.perimeter).





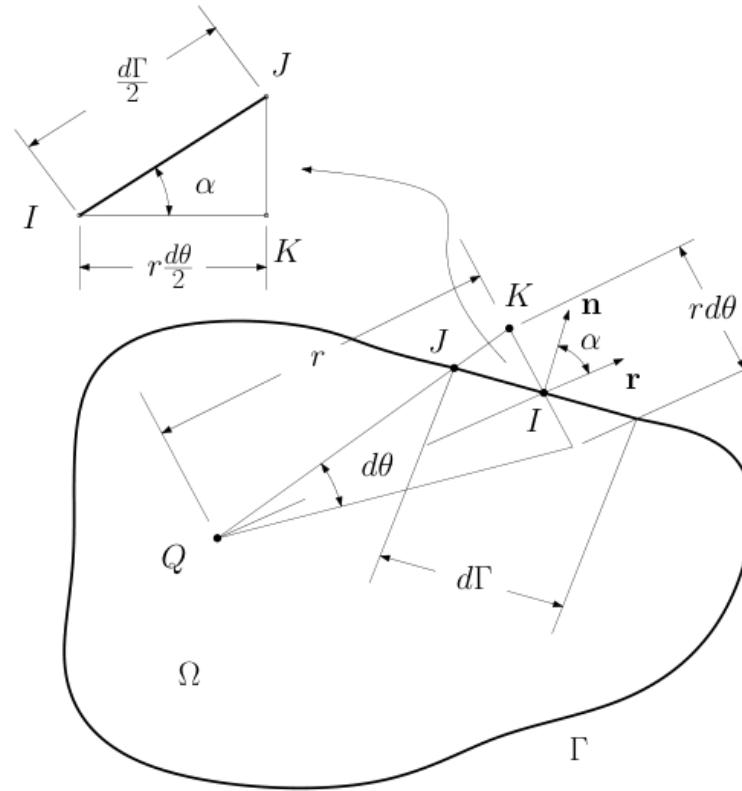
```
g  = geometric_props(dad)                                # strategy=:radial
g2 = geometric_props(dad; strategy=:divergence)
g3 = geometric_props(dad; npg_radial=20)
g4 = geometric_props(dad; npg_boundary=16)
```

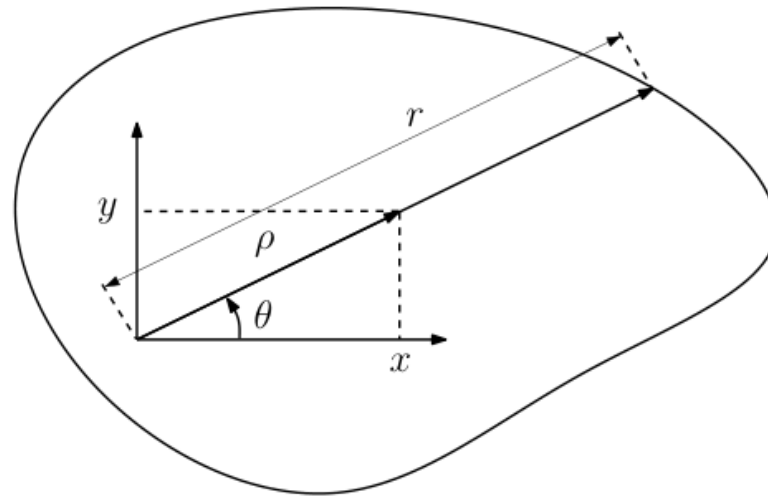
`npg_boundary=nothing` reutiliza `elem.Jacobian × dad.elem_weight`. Um inteiro reintegra cada elemento com `shapefun(dad.element_type, ξ)` e Gauss.

Integração radial (:radial, default)

Pólo na **origem**. $\mathbf{r} = \mathbf{x}$, $r = |\mathbf{r}|$, $\hat{\mathbf{r}} = \mathbf{r}/r$.

$$dA = \rho, d\rho, d\theta, \quad d\theta = (\mathbf{n} \cdot \hat{\mathbf{r}})(d\Gamma)/r.$$





Radial: F e o loop

$$I = \int_{\Omega} f, dA = \int_{\Gamma} F(\mathbf{x}) \frac{\mathbf{n} \cdot \mathbf{x}}{r^2} d\Gamma$$

$$F = \int_0^r f(\rho \hat{\mathbf{r}}), \rho, d\rho.$$

```
# GeometricProperties.jl - _geom_accum_2d_radial
# Fa, Fx, Fy = ∫_0^r {1, x, y} ρ dp      (_calc_F_2d)
# A += Fa * nr / r * wJ
# Sx += Fx * nr / r * wJ
# Sy += Fy * nr / r * wJ
# centróide = (Sx, Sy) / A
```

```
function _calc_F_2d(r, theta, qsi, w)
    dro = r / 2
    Fa = Fx = Fy = 0.0
    for i in eachindex(qsi)
        ρ = r / 2 * (qsi[i] + 1)      # ρ ∈ [0, r]
        x, y = ρ * cos(theta), ρ * sin(theta)
        Fa += ρ * dro * w[i]          # ∫ ρ dp (f = 1)
        Fx += x * ρ * dro * w[i]
        Fy += y * ρ * dro * w[i]
    end
    return Fa, Fx, Fy
end
```

Para $f = 1$: $F = r^2/2 \rightarrow A = \frac{1}{2} \int_{\Gamma} (n \cdot x) d\Gamma$. Com `npg_radial=12` a primitiva polinomial já é exata.

$$\int_{\Omega} \nabla \cdot \mathbf{F}, dA = \int_{\Gamma} \mathbf{F} \cdot \mathbf{n}, d\Gamma.$$

F	$\nabla \cdot \mathbf{F}$	Resultado
$(x, 0)$ ou $(0, y)$	1	$A = \int_{\Gamma} x n_x$
$(x, y)/2$	1	$A = \frac{1}{2} \int_{\Gamma} x \cdot n$
$(x^2/2, 0)$	x	$\int_{\Omega} x = \int_{\Gamma} (x^2/2) n_x$
$(0, y^2/2)$	y	$\int_{\Omega} y = \int_{\Gamma} (y^2/2) n_y$
$(0, y^3/3)$	y^2	I_x (moment_Ix)

```
P += wJ
A += 0.5 * dot(x, n) * wJ
Sx += 0.5 * x[1]^2 * n[1] * wJ
Sy += 0.5 * x[2]^2 * n[2] * wJ
```

npg_radial é ignorado. A outra fórmula ($A = \int_{\Gamma} x n_x$) está em area_divergence:

```
function area_divergence(dad::BEMdata)
    A = 0.0
    w_el = dad.elem_weight
    for elem in dad.elements, k in eachindex(elem.index)
        i = elem.index[k]
        wJ = elem.Jacobian[k] * w_el[k]
        A += dad.Nodes[i][1] * dad.Normal[i][1] * wJ
    end
    return A
end
```

```
g = geometric_props(dad)                                # :radial
gd = geometric_props(dad; strategy=:divergence)
Adiv = area_divergence(dad)
@show g.area gd.area Adiv
```

No scripts/circulo.jl as três áreas coincidem até o erro de quadratura de Γ (≈ 3.14110).

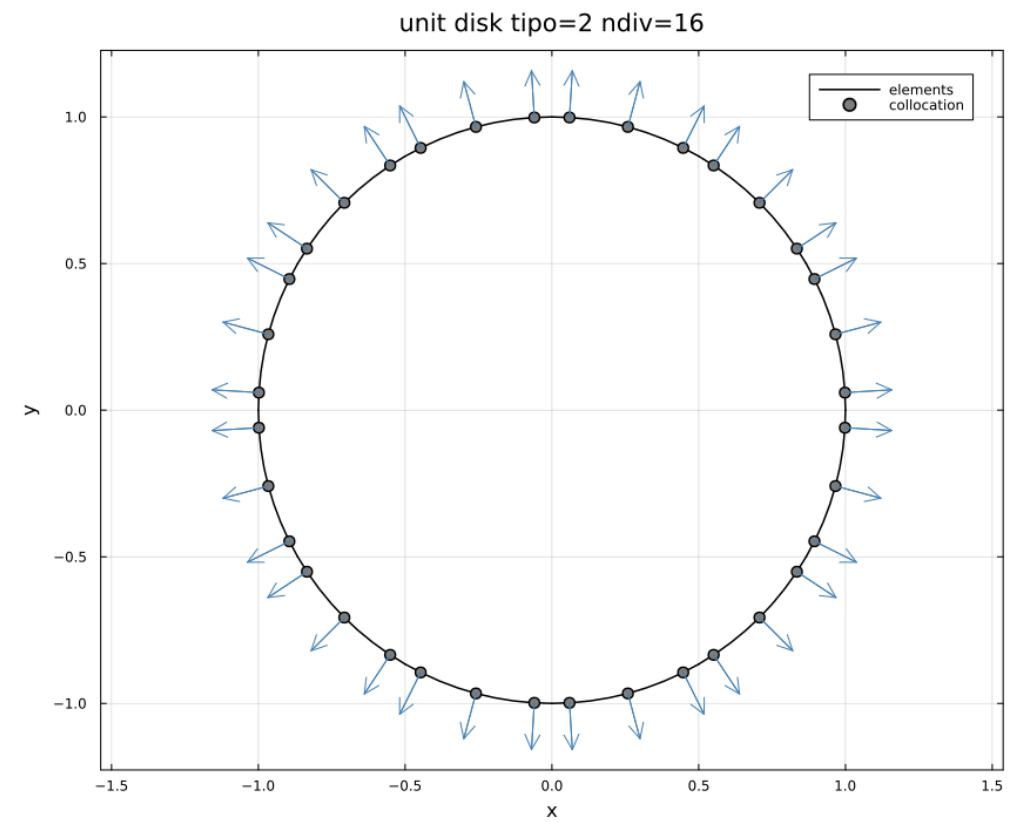
Radial × divergência. Para $f = 1, x, y$ coincidem (primitiva polinomial + mesma quad. em Γ). Radial generaliza f via F . Divergência é imediata quando existe $\mathbf{F}$ polinomial simples (moment_Ix, etc.).

Exemplo 2D: disco unitário

Arquivo scripts/circulo.jl. Quatro arcos, ordem 2, 16 elementos 1D.

```
msh = circulo(; ndiv=16, show=false, nome="ex_circulo", ordem=2)
dad = format2d(msh, Laplace(1.0); tipo=2, pontointerno=false)
g = geometric_props(dad)
plot_geo(dad; show_normals=true, title="unit disk tipo=2 ndiv=16")
```

Exemplo 2D: disco unitário



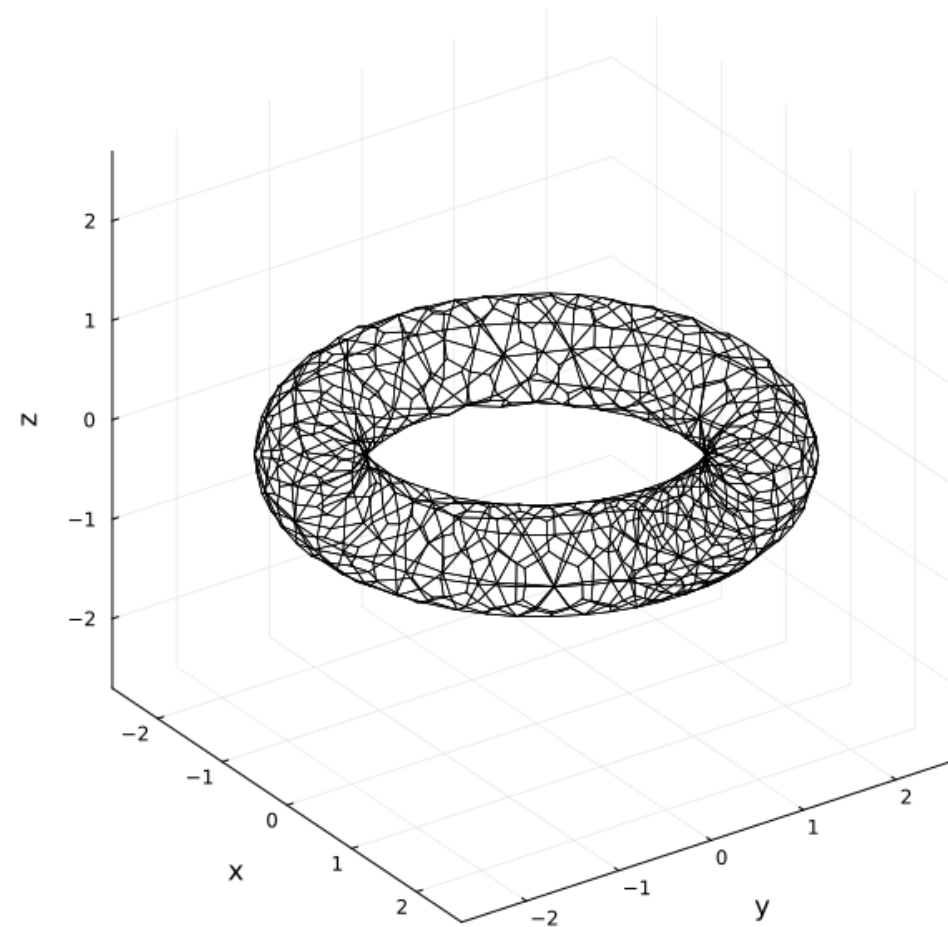
Exemplo 3D: toro — scripts/toro.jl

A **mesma** API vale em 3D: gerador $\rightarrow$ format3d $\rightarrow$ geometric_props.

toro chama `gmsl.model.occ.addTorus` (raio maior R , tubo r , eixo z , origem).

$$S = 4\pi^2 Rr, \quad V = 2\pi^2 Rr^2, \quad \mathbf{c} = \mathbf{0}.$$

```
R, r = 2.0, 0.5
msh = toro(; nome="ex_toro", R=R, r=r, lc=0.25, show=false)
dad = format3d(msh, Laplace(1.0); tipo=1, pontointerno=false)
g    = geometric_props(dad)                                # :radial
gd   = geometric_props(dad; strategy=:divergence)
plot_geo(dad; show_normals=false, show_nodes=false)
```



Grandeza	Numérico	Exato
área de superfície	39.18	$4\pi^2 Rr \approx 39.48$
volume	9.57	$2\pi^2 Rr^2 \approx 9.87$
centróide	$\approx (0, 0, 0)$	$(0, 0, 0)$

Radial = divergência até roundoff. 3D: $V = (1/3) \int_{\Gamma} x \cdot n$; radial usa `_calc_F_3d` e $j_{\text{fac}} = (n \cdot x)/R^3$.

$$\mathbf{n} \parallel \partial_{\xi} \mathbf{x} \times \partial_{\eta} \mathbf{x}, \quad J = | \partial_{\xi} \mathbf{x} \times \partial_{\eta} \mathbf{x} | .$$

```
N, dNx, dNy = shapefun2D(Equispaced(1), qsi)
nodes[idx]   .= N * X
dx1, dx2     = dNx * X, dNy * X
J            = norm.(cross.(dx1, dx2))
normal[idx]  = cross.(dx1, dx2) ./ J
```

Faces Gmsh: só quads (tipo 3 linear ou tipo 10 quadrático). cubo + format3d: $S = 6$, $V = 1$, $c = (1/2, 1/2, 1/2)$.

Boas práticas de malha

1. Contorno externo anti-horário; furos horários.
2. Elementos descontínuos no campo (padrão format2d).
3. ordem da malha Gmsh alinhada a tipo em format2d.
4. Refino perto de cantos e furos (placa_com_furo).
5. Nesta aula: pontointerno=false basta para prop. de Γ .
6. Em 3D, volume no .msh ajuda a orientar **n** (toro tenta generate(3)).

Exercícios — entrega

Scripts no ambiente GeomGmsh (`julia --project=codes/geom_gmsh`), `plot_geo` quando fizer sentido, erro vs analítico e 2–3 frases.

Fontes: `src/{Input,Interpolation,GeometricProperties,Meshes}.jl`, `scripts/circulo.jl`, `scripts/toro.jl`. Soluções em `_solu/`.

E1 — shapefun e partição da unidade

Para $p \in \{1, 2, 3, 4\}$:

- (a) `poly = Legendre(p)`: em 21 abscissas, $\sum_i N_i = 1$ e $\sum_i N_{i'} = 0$.
- (b) Plote as colunas de $N(\xi)$ para $p = 2$ e $p = 3$.
- (c) `Equispaced(2)` em $\xi \in \{-1, 0, 1\}$ vs Lagrange clássico.
- (d) `nodes(Legendre(p))` vs `discontinuous_nodes_weights(p)[1]` — são o quê? (`weights(poly)` são baricêntricos; o segundo retorno é a quad. de Gauss.)

E2 — coroa circular (anel)

$1 \leq r \leq 2$. Exato: $P = 6\pi$, $A = 3\pi$, centróide na origem.

(a) `ndiv` in $\{8, 16, 32\}$, `tipo` in $\{1, 2\}$: erros de P e A .

(b) No **mesmo** `dad`: `:divergence`, `area_divergence` vs `:radial`.

(c) `npg_boundary=16` e `npg_radial=20` vs default.

(d) `plot_geo`: normais no furo para fora de Ω ?

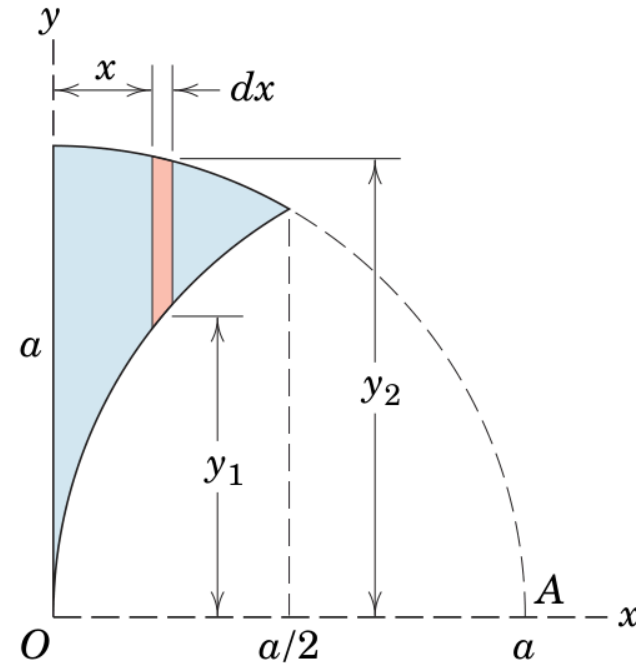
(e) Tabela `ndiv`, `tipo`, `n_col`, `e_P`, `e_A`, `e_Adiv`. Quem atinge $e_A < 10^{-3}$ com menos nós?

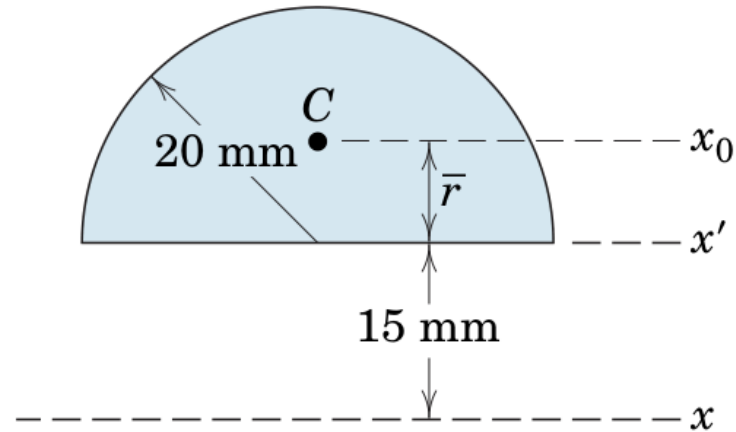
E3 — círculo unitário, ordem em n

círculo com $n \text{div} \in \{8, 16, 32, 64\}$, $\text{tipo}=1$ e $\text{tipo}=2$.

P e A vs 2π e π . Qual ordem aparente em n (ou $h \sim 1/n$)?

Lembrete: $\text{tipo}=1$ é o n -ágono regular.





$$I_x = \int_{\Omega} y^2 dA \text{ via } \text{moment_Ix} (F = (0, y^3/3)).$$

Fórmulas da lista; geradores: `triangulo_equilatero`, `triangulo_com_incirculo`, `setor60`, `semicirculo`.

E5 — furo e orientação

`placa_com_furo.`

(a) $A = A_{\text{ret}} - \pi R^2, P = P_{\text{ret}} + 2\pi R.$

(b) `reverse_hole=false` e `format2d(; orient=false)`: sinal de A .

(c) `plot_geo`: normais no externo vs furo.

Extra 3D: cubo ($S = 6, V = 1$) ou `scripts/toro.jl`.

O que fica para Laplace 2D

- Convenção " $\theta; T$ ", " $1; q$ " (e elasticidade $t_x; u_x; t_y; u_y$).
- `attach_analytical!`, `H_G_*`, `solve`, `rel_error`.
- CDC em cantos e o papel do elemento descontínuo.

Leituras e código

- `codes/geom_gmsh/scripts/circulo.jl`, `scripts/toro.jl`
- `src/Input.jl` — `format2d`, `format3d`, `discontinuous_nodes_weights`
- `src/Interpolation.jl` — `shapefun`, `shapefun2D`
- `src/GeometricProperties.jl` — `:radial`, `:divergence`, `moment_Ix`
- `src/Meshes.jl` — `circulo`, `anel`, `placa_com_furo`, `toro`, `cubo`
- `_solu/` — E1 a E5 resolvidos